

Controller-Preserving Dynamic Update Synthesis from Local Contracts

ANONYMOUS AUTHOR(S)

Verified controllers leave the update between them unspecified. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) generates the intervening behavior from local transfers and requirement lifetimes for fixed endpoint controllers. We prove that the generated game represents all contract histories and preserves the selected new controller's continuation. Lazy synthesis returns a safe finite-completion certificate covering every old entry and every outcome, or proves impossibility. A mandatory-command obstruction identifies choices needed for feasibility; rolling batches of k among n replicas keeping m ready are feasible iff $k \leq n - m$. Five constructed families yield 33/55 fine-WIN/merged-LOSS comparisons. Lazy decides 26/27 adapted contracts within budget versus 14 with full construction; independent updates separate $n + 1$ from 2^n explored states under the default order. Guarantees concern declared scopes and finite event executions with quiescent commands/handover; deployment requires a conforming adapter.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Controller-Preserving Dynamic Update Synthesis from Local Contracts. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 20 pages.

1 Introduction

Verified endpoints leave their intervening update unspecified [3, 6, 12, 28]. An update may be requested while a workpiece remains in a cell: replacing empty stations must preserve that workpiece throughout the change. Replacement can therefore require ordinary movement and overlapping old/new obligations. Every old entry needs safe handover despite all outcomes; safety alone permits infinite execution.

Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) generates an update game from local replacements and requirement lifetimes (Figure 1). Synthesis returns a completing policy, rank and compatible new-controller state, or a losing region. This separates specifying obligations from enumerating mixed configurations; all queried outcomes remain represented.

1. Supply contract

Fixed controllers, local transfers, requirements and initializers

2. Generate game

Combine versions, monitor memories, commands and targets

3. Search lazily

Query events as needed; retain all outcomes of each query

4. Check result

WIN: policy, rank, handover; LOSS: losing region

Fig. 1. From local specifications to a certified update. Game successors are generated on demand during search. WIN covers every old entry and outcome; requirement meaning and execution conformance have separate proof obligations.

Dynamic Update Controller Synthesis (DUCS) synthesizes update and subsequent control from global transfers and transition requirements [24, 25]. Meeting a new specification can still remove supplied-controller behavior. FG-DUCS preserves every continuation of the selected new-controller state, which need not be its original initial state. A constructive Cell encoding relates the interfaces (Section 3.4); established finite-game reasoning solves the generated contract.

C1: Controller-preserving construction. We prove that composing local transfers and requirement lifetimes preserves contract histories and the selected endpoint’s continuation behavior (Section 3).

C2: Certified synthesis. We give a terminating, sound and complete lazy procedure covering all entries and outcomes, with checkable WIN/LOSS certificates and a conditional connection to execution (Sections 4 and 5).

C3: Structural boundaries. A mandatory-command obstruction explains merged losses; Rolling gives an exact threshold, and independent updates separate default-order $n + 1$ from 2^n states (Sections 4.3 and 6.2).

C4: Traceable evidence. Source-to-monitor and handover checks accompany 27 adapted contracts and 55 controlled comparisons, including 33 granularity separations (Sections 5.1–6.4).

The evaluation follows three obstacles: retaining feasibility with local rather than merged commands (RQ1), safe finite completion from every entry and outcome (RQ2), and avoiding full mixed-state construction (RQ3).

Commands and handover require uncontrollable (UC) quiescence. Guarantees cover finite-state safety lifetimes and finite completion; old liveness, service rates and unvalidated deployment behavior are excluded.

2 From a Complete Contract to a Completing Policy

Component granularity chooses what is replaced together; requirement granularity chooses which guarantees start or end together. Two contracts separate these choices.

2.1 Cell: Complete Local Input and Whole Policy

Consider a constructed two-station cell with exactly one workpiece [19, 25]. Both transfers require an empty station. A keeps its behavior; new B requires inspection before departure, while the old rule forbids inspection. Ordinary events move or inspect the workpiece. New-start before old-stop prevents an unprotected gap; a pending inspection waits for old-stop. Inspection can repeat safely forever.

Table 1 specifies every component, endpoint and monitor. The interval preserves exactly one workpiece. All labels are controllable; new-start before old-stop is the only dependency. Transfers are $g_A = g_B = \{(e, e)\}$, where e means empty.

Endpoint tuples list A, B , controller and its requirement monitor. The old initial tuple is $x_A = (h, e, c_0, k)$ and its other reachable tuple is $x_B = (e, h, c_0, k)$; both are entries. The new initial tuple is $z_A = (h, e, c_0, c)$; its other reachable tuples are $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Only $\{z_A, z_B\}$ are loadable targets: inspection must be clear at handover. The interval initializer records the holder in either all-old tuple. The new initializer admits all eight version-tagged one-workpiece tuples, installing d if B holds and c otherwise. These data specify the contract without an update sequence.

Feasibility and completion. Moving the workpiece enables local replacements; joint replacement requires both stations empty, impossible under any move or boundary. Endpoints and requirements remain fixed. Inspection may repeat forever safely, but every policy arrow decreases rank from six or five to zero, proving all-outcome handover from both entries.

Table 1. Complete Cell components, controllers and monitors. o/n denote old/new versions; h/e mean holding/empty. Parentheses mark initial states. m_A, m_B move to A, B ; j inspects at B .

Object	States (initial)	Ordinary transitions
A^o, A^n	$h, e (h)$	$h \xrightarrow{m_B} e, e \xrightarrow{m_A} h$
B^o	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e$
B^n	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e, h \xrightarrow{j} h$
C^o	$c_0 (c_0)$	$c_0 \xrightarrow{m_A, m_B} c_0$
C^n	$c_0, c_1 (c_0)$	$c_0 \xrightarrow{m_A, j} c_0, c_0 \xrightarrow{m_B} c_1, c_1 \xrightarrow{j} c_0, c_1 \xrightarrow{m_B} c_1$
t_{one}	$A, B, \text{err} (A)$	$A \xrightarrow{m_B} B, B \xrightarrow{m_A} A, A \xrightarrow{m_A} \text{err}, B \xrightarrow{m_B} \text{err}$
t_{old}	$k, \text{err} (k)$	$k \xrightarrow{j} \text{err}$
t_{inspect}	$c, d, \text{err} (c)$	$c \xrightarrow{m_B} d, d \xrightarrow{j} c, d \xrightarrow{m_A} \text{err}$

k, c, d mean old-safe, inspection-clear and inspection-pending; c_0, c_1 are distinct controller states.

Component/controller alphabets are exactly their displayed labels; unlisted edges are absent. Unlisted non-error monitor edges self-loop, and err absorbs every label. Each comma-separated label denotes an edge.

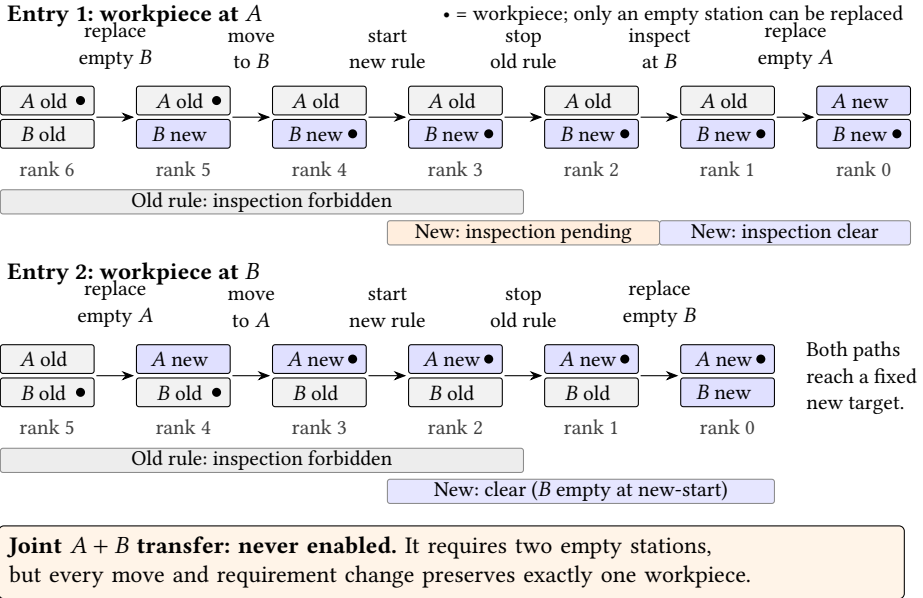


Fig. 2. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward a fixed new target. Joint transfer merges these local commands with endpoints and requirements unchanged; this is not a DUCS encoding.

A whole policy from the local input. Figure 2 covers both entries, all outcomes and both targets: 13 policy states and 11 edges. Shown events have singleton outcomes; other controllables are disabled. On the first path, activation creates a pending duty that ordinary inspection clears before handover

to z_B ; the second reaches z_A . Thus the new monitor affects ordinary update behavior. Section 5.1 explains its memory; Technical Appendix B gives symbolic paths and S1 their closure.

2.2 Policy: Scheduling Requirement Lifetimes

Opposing requirement boundaries. Policy changes auditing and access rules. At least one audit must remain active, while the old and new role requirements must not overlap. Figure 3 shows its complete saved policy; grouping both starts and both stops makes both entries lose, and the saved losing closure checks all alternatives. Technical Appendix I gives the complete finite input and boundary argument.

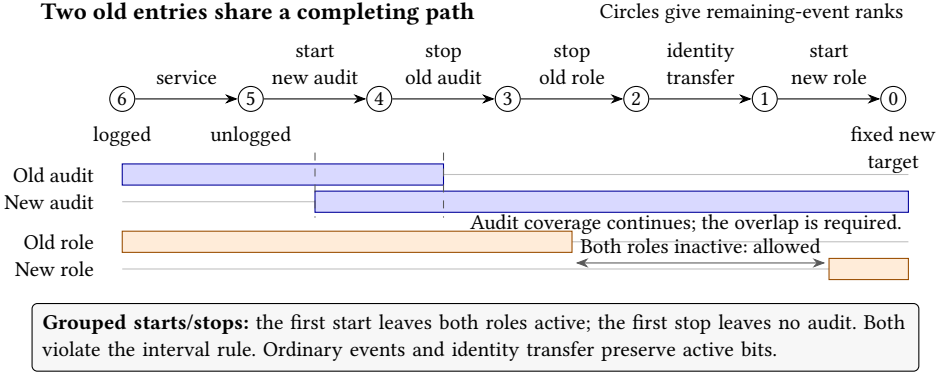


Fig. 3. Policy with no supplied precedence. Logged/unlogged mark both entries; bars mark active requirements. New-audit start resets to unlogged; handover reaches new audit unlogged and new role valid. Arrows have one action and outcome. The role gap is allowed; ordinary events may be withheld, with no service-continuity guarantee.

The Policy contract. Both one-state runtimes have identity transfer and controllable log_o , log_n , $service$, $access_o$ and $access_n$ loops. Each audit starts unlogged: its log marks logged, service requires logged then resets it. Each role starts valid and rejects the opposite version's access. Other monitor labels stutter; errors absorb. Initially $(a_o, a_n, r_o, r_n) = (1, 0, 1, 0)$; always $(a_o \vee a_n) \wedge \neg(r_o \wedge r_n)$. Endpoints alternate their own log/service and allow their own access; both audit-memory states with valid role are old entries and new targets.

3 Defining an Update Contract

Seven supplied groups determine the game: *event control*, *fixed old/new controllers*, *components and local transfers*, *old/new/interval requirements*, *activation initializers*, *optional command precedence*, and *handover targets*. They do not prescribe a sequence. Section 5.1 connects initialized monitor states to declared requirement scopes.

3.1 Supplied Choices

Finite LTSs and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{uc} \uplus \Sigma_c$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [20]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for

violating prefixes [1, 18]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* t_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed; in the cell, initialization records whether inspection is pending. For history-based requirements, residual soundness (RS) requires every continuation accepted by the installed state to be safe after every declared activation history. Activation-scoped invariants instead retain current facts and constrain behavior from their start; both interpretations are defined in Section 5.1. For $r \in R^{\text{upd}}$, $\text{dom } t_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i \Sigma_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states. FG-DUCS keeps the supplied new controller fixed. The nine adapted benchmark inputs admit all of $\text{Reach}(\text{CL}^{\text{new}})$; constructed contracts may select a proper subset, as Cell does. For a supplied maximally permissive controller this is its reachable winning region, equal to the full region only if all winning states are represented and reachable.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome. They require uncontrollable quiescence: no modeled uncontrollable event may be enabled when an update command is issued.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. Eligibility requires finite, consistently typed states/events and controllability; total deterministic monitors with absorbing errors; typed transfers and non-error initializers covering interval entry; and strict partial-order dependencies. Endpoints are safe, deadlock-free and uncontrollable-preserving, with controller alphabets covering environment labels; targets are nonempty and reachable. Justifying initializers against their declared requirement meaning is separate from game eligibility. Interval and post-handover duties belong to R^{upd} and R^{new} ; $\text{start}(r^{\text{new}}) < \text{stop}(r^{\text{old}})$ enforces overlap.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i}^v$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states. It initializes interval monitors through $\iota_r(\xi)$, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The policy replaces C^{old} , discarding its state and scheduling restrictions; undeclared objectives, including old liveness, are not preserved. Old monitors remain until stopped. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage avoids waiting for a favorable old state; an admission predicate would define a different contract. In the cell, $Q_0 = \{q_A^0, q_B^0\}$ distinguishes the two holders.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

This global *uncontrollable quiescence* differs from transaction quiescence [17] and tranquility [32]. It is a command restriction, not a consequence of local transfer. Ordinary controllables may coexist with uncontrollables; the UC-priority lemma later justifies suppressing them. $\text{Post}(q, a)$ contains exactly the following outcomes; unmentioned coordinates remain unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } \iota_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = \iota_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

On the cell path starting with A holding, start preserves old/empty A and new/holding B , installs inspection-pending in the new monitor and removes itself from P . The old and interval monitors stay active.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals.* An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

At a goal, all components and endpoint monitors are new; interval obligations end and matching new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. No uncontrollable can compete. A matching UC-enabled state is not a goal; compatible targets may differ in later safe behavior.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. An uncontrollable self-loop blocks updates by Equation (3) and permits a losing infinite run. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_{\sigma}(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts observed transitions and assumes no fairness.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_{\sigma}(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_{\sigma}(q)$.

The FG-DUCS problem. The seven supplied groups define the eligible game. Decide whether one policy safely completes it from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A *local update history* records labels and realized local outcomes from an old entry, independently of a global game. It obeys local transitions, active monitor folds, one-shot commands, precedence, quiescence and target matching. Old control is released at entry (full operational definition in Technical Appendix A).

PROPOSITION 3.1 (ADEQUACY OF THE GENERATED UPDATE GAME). *For an eligible contract, map each local update history to its current component tuple, active monitor memories and unexecuted commands. Histories with the same image admit the same next events and all the same local outcomes, agree on safety, and have the same handover targets. This map gives a label- and outcome-preserving correspondence between local update histories and paths of the generated game. A history-dependent policy satisfying the source obligations exists exactly when a winning policy exists in that game.*

The proof follows each local operation in both directions and transports policies while preserving outcomes, safety, deadlocks and completion (Technical Appendix A). This concerns the supplied contract; requirement meaning and Java frontend correctness are separate obligations. The construction connects each reached goal to the supplied endpoint without resynthesizing its subsequent control. The following preservation property follows from that explicit connection.

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned WIN certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions,

without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies. Execution additionally requires the adapter premises in Section 5.

Why continuation preservation is a separate obligation. After completion, take a supplied new closed loop with controllable self-loops a, b and goal *true*. Disabling b remains legal, deadlock-free and satisfies that goal [25, Defs. 3.9–3.10 and 4.3], but removes all continuations containing b . Reconnecting preserves both. The Cell correspondence below preserves continuations by construction; new-goal satisfaction alone does not imply preservation.

Table 2. The same Cell obligations through two construction interfaces. The native column describes our explicit DUCS encoding [25, Defs. 4.1–4.3], not a lower bound on other encodings.

Obligation	Explicit native Cell modeling	FG-DUCS input and generated structure
Replace empty stations	Eight mixed stages record versions and holder; transfer guards require empty stations.	Component models and local empty-station transfers generate mixed configurations and guarded edges.
Preserve inspection duties	Holder-refined old control, history variables and temporal requirements encode memory and boundaries.	Typed monitors, initializers and ordering generate boundary events, active memories and lifetime guards.
Preserve fixed continuation	A supplied continuation graph, guarded handover and eventual-handover requirement connect fixed control.	Fixed controllers and targets generate compatibility conditions; synthesis selects the load state.

Native DUCS also generates its standard protocol and interrupt composition, and synthesizes subsequent control. Both methods require justified input meanings. The Cell proof preserves the displayed paths and subsequent traces; it establishes neither general equivalence nor measured effort savings.

3.4 A Constructive Correspondence for Cell

Native DUCS can express the two Cell paths [25, Defs. 4.1–4.3 and 5.1]. Its new environment contains eight mixed version/holder stages plus three new-endpoint states. Ordinary moves change the holder, empty-station transfers change one version, and a guarded handover connects all-new stages to clear-inspection targets. Holder refinement, history fluents and transition requirements enforce lifetimes. Technical Appendix K proves path and continuation preservation; Table 2 contrasts these explicit inputs with FG-DUCS’s generated structure. The 11 environment states exclude protocols/monitors and imply no minimum cost. Eventual handover uses theoretical FLTL beyond native safety- T [25, Sec. 5.3]; this is a construction, not a native solver run.

4 Synthesizing and Checking an Update Policy

Lazy explores products on demand using finite reachability reasoning [7, 8, 31].

A *bucket* is $\text{Post}(q, a)$; an empty bucket disables a . At a UC-enabled state, disabling controls retains a successor and only removes runs; a winner already handles every UC outcome. This preserves winning (Lemma A.1).

4.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [5]. It tracks discovered states V , states $X \subseteq V$ with all UC candidates queried, and proved winners W . States in X may retain unqueried controls; W may omit winners.

Delaying other control events retains all outcomes of the proving choice (Figure 4).

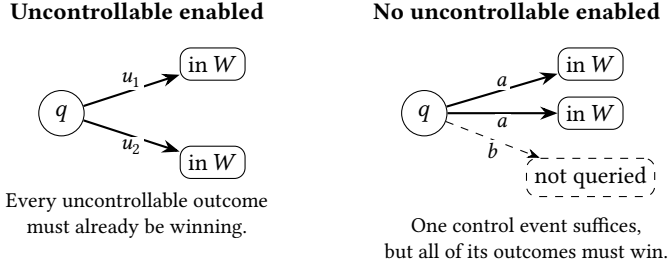


Fig. 4. Safe promotion to W . The two a edges are nondeterministic outcomes of one control event, not two selectable actions. Other control buckets may be postponed.

Expansion queries every UC candidate. If none is enabled, it queries controls through the next nonempty bucket or exhaustion; Retry resumes unfinished cursors. Each query retains all outcomes. Candidates include local events and precedence-enabled commands in any fixed finite order.

Promote. A safe expanded state wins when its nonempty proving set—all UC successors, or one control bucket at quiescence—lies in W ; its rank is one above their maximum. Reverse incidences trigger predecessor reconsideration. Failure requires exhausted candidates and a promotion fixed point. Once all entries win, extraction selects one lower-rank control bucket at each reached quiescent non-goal, none elsewhere; it need not follow promotion or minimize length.

4.2 Correctness and Certificates

State-based policies suffice for this finite game; completeness also excludes history-dependent winners on losing contracts (Technical Appendix A).

THEOREM 4.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

Why the guarantee holds. Every discovered safe non-goal is either waiting for expansion or has all UC candidates queried; every expanded unproved quiescent state with candidates left remains in Retry. Promotion inserts only safe states with a complete nonempty proving set in W , assigning a larger rank. These invariants give safe, nonblocking finite completion on success. At failure, exhausted queues and promotion leave an entry in $L = V \setminus W$: every safe state there has a UC outcome in L , or every enabled control has an outcome in L . The environment can therefore defeat every policy, including history-dependent ones. Finitely many states/candidates ensure termination. Technical Appendix A gives the full proof and monitor-interval induction.

Algorithm 1: Lazy**Input:** Typed update contract (Section 3)**Output:** (σ, rk, κ) or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $rk(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Add queued goals to  $W$  once with rank zero; exhaust Promote, retaining witnesses/ranks, removing winners
   from Retry and propagating predecessors;
5   if  $Q_0 \subseteq W$  then return  $(\sigma, rk|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
   choices;
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
8     Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or
     exhaustion;
9     Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
10  else
11    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
12    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
13  Queue newly discovered goals for rank-zero insertion; put newly discovered safe non-goals in Open;
```

Checking the returned evidence. For WIN, let $D_\sigma = \text{Reach}_\sigma(Q_0)$. The checker requires exactly D_σ as the rank domain, safety throughout it and, at every non-goal, enabled selected controls, nonempty successor sets inside it, and strict rank decrease for every outcome. At goals it checks rank zero, UC quiescence and compatible $\kappa(q) \in Z_{\text{load}}$, with $\text{dom } \kappa = D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty.

For LOSS, $L \subseteq \mathcal{Q}$ must include an entry state and exclude all goals. Every safe $q \in L$ must have some UC outcome in L when UC is enabled; otherwise every enabled controllable must have an outcome in L . Unsafe states and non-goal deadlocks lose. These conditions certify search relative to shared Post and endpoint predicates [16]; the implementation and independent checks in the main paper state the trust boundary (Section 6).

4.3 Why Local Choices Need Lazy Exploration

With indexed predicates and constant-time bookkeeping, output-sensitive exploration is linear in discovered states, queried candidates and returned outcomes. This excludes endpoint preprocessing and Java's sorting/entry scans. Lazy explores a subgraph of Direct-Full but may reach its whole graph (Technical Appendix E; S2).

An exponential separation family. For $n \geq 1$, take n components with one old state, one new state and singleton transfers. Each version and endpoint has a controllable *tick* self-loop. No monitors, precedence or UC events occur; the unique new endpoint is the target. Every transferred subset is reachable: Direct-Full discovers 2^n states. Lazy orders transfers before *tick* and *Open* before *Retry*, discovering one n -step chain and its goal. Ranks $0, \dots, n$ propagate before alternatives are retried: exactly $n + 1$ states. Self-loops cannot prove completion. Both use the same goal; the comparison concerns explicit exploration, not symbolic representations.

5 Interpreting and Executing a Winning Policy

5.1 Giving Requirements Their Intended Lifetimes

Cell’s new duty covers the workpiece already at B , requiring occupancy-aware activation (Table 1; Figure 2). The activation-safe history checks (A) below pose a different question.

For Cell’s start-only reference, initializing inspection as clear is exact, even while B holds an uninspected workpiece. Initializing as pending is conservative for that reference. The duty actually needed at activation is therefore specified and proved separately:

An exact activation obligation with ordinary work. For Cell, define an owed-inspection bit at activation to be 1 exactly when B holds the workpiece. Thereafter m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Encoding 0 by c and 1 by d makes every non-error monitor transition identical to this obligation’s rule, and precisely the forbidden departure enters error. Initialization and induction on the active suffix therefore establish exact enforcement of this activation obligation. In Figure 2, the first path starts with bit 1 and ordinary inspection clears it; the second starts with bit 0. This is an explicit duty beginning at activation, stronger than the start-only reference scope above. It does not extend the 138 A checks to actual histories or to other requirements.

Obligations that begin at activation. ACT starts a pure invariant with actual post-boundary observer values; it checks activation and the subsequent active suffix without counting earlier violations. Exact continuation language from a safe reference history with the same observer vector suffices (Proposition M.1, Technical Appendix M). The GSM argument below and Section 6.2.1 supply direct source-based cases.

A source requirement constraining ordinary update actions. GSM changes decoding from mode 2 to mode 3. From every old entry, follow its cycle to the first received-message state; then stop the old requirement, start the new one, decode in mode 3, output and transfer at the sender state. The current action observer is zero at activation, so the saved lookup installs non-error state zero. The active new monitor rejects mode 2 while permitting mode 3 and output; transfer reaches the supplied endpoint’s initial projection. These steps respect R1’s ban on ordinary actions between stop and start. A decreasing rank bounds completion by twelve events (Proposition M.2, Technical Appendix M). This analytical policy uses the existing source and lookup, not a new solver run. It establishes ACT over ordinary update behavior; the separate 45-pair source check establishes the interval’s E initialization. Earlier mode-2 events still preclude a whole-history claim.

When prior histories are part of the requirement. A safe monitor state must express a declared requirement meaning. Residual soundness (RS) requires every accepted continuation to be safe after every admitted activation history; equality is exact, while strict inclusion can reject feasible updates. An empty history scope passes vacuously, and synthesis does not establish scope completeness or intent. Technical Appendix B.1 gives the full definition.

A Boolean event-history variable, or *fluent*, records setting/resetting labels. The evaluation distinguishes three history scopes:

A (activation-safe), for new requirements (NEW): admit only safe pre-activation histories consistent with the observed component state.

E (entry-scoped), for interval requirements (UPD): start the reference at update entry.

H (history-inclusive): count pre-activation violations too; not claimed. Synthesis does not establish whether the chosen scope matches intent (S3). A inclusion alone does not establish ACT; the activation arguments above and the thirteen PC2 obligations use their actual source meanings.

5.2 Execution and Handover

The execution connection has four distinct premises (Technical Appendix H):

A1: endpoint observations preserve component/monitor states and same-labeled endpoint edges.

A2: update control disables only unselected controllables, preserves every UC event/outcome, reports the successor and eventually observes a next event.

A3: commands start quiescently and finish atomically, finitely and failure-free with an observed modeled successor.

A4: entry observes/copies the actual old state; quiescent handover installs the selected compatible state, using finite failure-free linearizable boundaries [14].

Under A1–A4, every execution maps to the linked model and reaches handover within the entry rank (Theorem H.1). Complete activation histories and RS additionally justify history-based interpretations. No event-choice fairness is assumed. Internal divergence and unobservable outcomes are excluded; rank bounds events, not time. Table 3 separates these premises from the scope checks and proofs actually obtained.

Table 3. What the guarantees establish and where their evidence ends. The three layers are game correctness, requirement interpretation and execution.

Layer/scope	Established result	Evidence and remaining obligation
Game	Safety, all-outcome finite completion and compatible handover from all entries	Theorem 4.1; checking trusts supplied successor/endpoint predicates
A: safe history	NEW language included in every admitted safe-history residual	138 occurrences (46 requirements, three variants); nonempty scopes unverified beyond one PC2 tuple
E: from entry	UPD initialization equals the entry residual	135 occurrences: 46 R1/89 R2; guards/stage fluents justify entry values
ACT: from start	Source invariants hold from activation through handover and continuation	PC2: all 13 new requirements, all 126 entries; no active ordinary event. GSM: all entries with decode/output
Cell duty	Actual occupancy initializes inspection debt; the monitor exactly enforces its update rules	Bit/monitor induction in Section 5.1; both entries, including ordinary inspection
Execution	Observed runs follow policy edges and finish within entry rank	Conditional A1–A4; adapter conformance and real-time bounds unvalidated

H (whole history) is not claimed: only 39/126 PC2 entries admit a safe full prefix; 87 do not. None of these checks establishes application intent.

6 Implementation and Evaluation

6.1 Design and Implementation

Four same-game methods use 27 contracts from 9 inputs. Legacy, our traditional DUCS/GR fork, instead synthesizes subsequent control. Base has no interval requirement: its policy may stop all old requirements before starting any new one. R1/R2 constrain that gap: R1 restricts ordinary actions (Railcab bans braking); R2 constrains boundaries versus transfers (GSM’s decode boundaries precede ENV transfer). PC2-Rolling requires continuous availability. Gap lengths and suppressed ordinary events are unmeasured.

Implementation. Our Modal Transition System Analyser (MTSA) extension [9] checks certificates [21]; Link checks old–update–new composition, entries and targets. An independent oracle

parses restricted Finite State Processes (FSP). Finite-JSON runs save losing membership; FSP benchmark runs report only its statistics. Runtime conformance is unchecked.

Use of generative AI. Generative AI assisted research design, implementation, tests, experiments, analysis, proofs and writing. Editorial reviews used Claude Opus 5.5 and ChatGPT Pro/Latest, not human evaluation. Authors checked outputs using tests, certificates, oracles and primary sources and retain responsibility; correlated faults remain possible. Technical Appendix N records models, dates and roles.

Constructing the benchmark inputs. Nine inputs—GSM, Industry, MetaSocket, PC1, PC2, PowerPlant, Railcab, Surveillance and Workflow—adapt eight DUCS sources; PC1/PC2 share a parent. Base/R1/R2 vary only interval requirements. Adaptations make PC tool results UC, shorten Railcab transfer paths while preserving both outcomes, and omit PowerPlant’s initial ENV transfer pair. They also change safety and ordering; whole-problem equivalence is unestablished (Technical Appendix J).

MTSA’s GR (generalized-reactivity) backend fixes endpoints in each JVM. The frontend derives boundaries and initializers. New initializers read current fluents; interval initializers read post-entry values. Targets include all reachable new-endpoint states. The first component stores global observers; nonlocal labels stutter its plant, and commands/transfers preserve observers. PC1/PC2 have one/two arms.

Inputs have 1–5 components, 4–131 entries and 3–26 commands (Technical Appendix C). Only Railcab transfers branch; none declares precedence. R2 encodes order in monitors. GSM/PC1’s single transfer makes merging the identity.

Exploration variants. All methods share entries, successors, goals, checks and terminals. The on-the-fly (OTF) variants omit controls at UC-enabled states and stop when all entries win. Direct-Full first materializes reachable buckets including those controls, then solves with the same UC-priority rule. Full+UC is an auxiliary full-construction variant that omits those controls during construction. At quiescence Lazy queries individual buckets, Eager all controls, and Update-first favors updates, testing O-DUCS’s priority [15] within our solver. Lazy favors stops/transfers, prioritizes states by endpoint mismatch and pending-command counts, and uses LIFO exploration before FIFO retries (Technical Appendix E); this order need not minimize exploration.

Design and measures. RQ3 runs sequentially on one Windows/Xeon host (Technical Appendix N) with a 64 GiB heap and 1,200 s JVM cap: 27×4 same-game cells plus 27 Legacy references. Decisions receive four additional trials, failures none. Medians/min–max require five consistent completions; interruptions/follow-ups remain recorded.

Measures separate explored states/queries/transitions, policies, solver time and peak working set (Technical Appendix J.8). Solver includes checking; JVM adds frontend, startup, output and endpoints. Legacy uses its native timer. Mac pilots are excluded from timing; appendix structural checks retain their Mac provenance.

6.2 RQ1: Can Local Choices Change Feasibility?

Cell and Policy isolate component and requirement-lifetime obstructions (Section 2). We next examine availability mechanisms and controlled comparisons.

What is merged. Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ simultaneously permits all targets in $\prod_{i \in B} g_i(s_i)$; a boundary block removes or initializes its monitors at the same component tuple. Other active monitors compose member-label effects, checked to commute pairwise on every monitor/residual state. Mapped precedence must

remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply. These are comparison contracts, not DUCS encodings.

The following obstruction explains the Cell, Policy and PC2 merges directly; the rolling threshold then gives an exact quantitative boundary.

LEMMA 6.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .*

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled. Policy retains its initial active bits until either grouped boundary violates the interval. PC2-Rolling’s enabled joint transfer violates availability (Section 6.2.1). These results concern specified merges, not native DUCS expressiveness.

Mechanisms and scope. Readiness/rejoining gates motivate staged replacement [23, 29]. Rolling begins with n ready replicas and no spares; its interval requirement keeps at least m ready. Each transfer boots its replicas; each has one UC readiness event and no waiting loop. Service self-loops are controllable. The target is all-new and ready.

PROPOSITION 6.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

The 15 Rolling separations instantiate Proposition 6.2; the 70 measured threshold cells are separate. Canary bounds reported broken replicas by $n - m$, with UC healthy/broken reports and recovery: all 15/15 pairs separate. Pending reports do not count as reported failures; this bound is not a continuous service-availability guarantee. Restricting outcomes to healthy wins both ways; removing recovery loses both ways. DB-Rolling preserves availability by replacing secondaries first; its $m = n = 3$ no-slack control loses both ways. Threads has bounded worker queues: backpressure permits quiescence (both WIN), saturated UC arrivals prevent it (both LOSS). Rolling+Audit retains an old audit until its replacement starts; singleton boundary merging preserves behavior, transfer merging loses. Policy is defined in Section 2; Technical Appendix J.10 specifies the other finite inputs.

Answer. The main suite yields 33/55 fine-WIN/merged-LOSS comparisons, in Rolling, Canary, Policy, DB-Rolling and Rolling+Audit. These families share capacity constraints and additionally exercise recovery, boundary compatibility and role/quiescence. Threads supplies both-WIN and both-LOSS controls (Table 4). Each obstruction identifies choices lost by the specified merge. PC2 contributes no solver WIN/LOSS pair: its merged run times out. The source argument below separately proves merged impossibility.

Table 4. Constructed granularity comparisons: each pair fixes one model and compares its fine and merged contracts. Policy merges boundaries; other rows merge transfers. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. The main total excludes Cell.

Family / control	Parameter domain	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
Rolling	$n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	$n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	2 rule pairs	1	1	0	0	0
DB-Rolling	$n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	$n = 2, m = 1$; 1 audit pair	1	1	0	0	0
Threads (control)	$n = 2 \dots 6$; queue 1,2	20	0	10	10	0
PC2-Rolling	2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	$n = 2 \dots 10$	9	9	0	0	0

Completed columns read fine / merged; Inc. denotes unresolved. PC2's merged solver remains TO, not LOSS. Threads is a negative control; its queue capacities are 1 and 2, with both backpressure and saturated-offer regimes. Rolling+Audit counts transfer merging; boundary merging remains WIN and adds no pair here.

6.2.1 A Source-Derived Calibration Case. PC2-Rolling adds controllable calibration requests and UC completions to new operational states, and two local transfers into calibration counterparts. Original transitions remain; erasing calibration recovers the operational process, without whole-problem equivalence. Fixed endpoints retain the baseline requirements and add availability, giving 13 new starts. Endpoint availability observes calibration requests/completions; the interval duty additionally treats transfers as calibration entry and requires one arm outside calibration. Transfers preserve ordinary observers, and initializers read them. These meanings are supplied; no precedence or complete update sequence is supplied.

Generated update and certificate. FG-DUCS combines local transfers, requirement boundaries, UC completions and fixed endpoint matching. One saved path moves arm 1 into calibration, waits for its completion, then transfers arm 2 and waits again; transfers enter calibration directly, so no calibration request occurs on this path (Table A7 in Technical Appendix L). The certificate covers all 126 entries with 381 states, 522 edges and maximum rank 37. Every outcome decreases rank and keeps 1 arm outside calibration. Link checks old–update–new composition (893 states, 2316 edges); this replay supplies no timing.

Source obligations through handover. Arm 1 retains $\Box(out_1 \Rightarrow Drilled_1 \wedge Cleaned_1 \wedge Painted_1)$. Its initially false fluents are set by successful tool results and cleared by reset. The saved path reaches its start, the eighth of thirteen, without out_1 or successful tool results. Lookup installs state 0 with zero observers, so departure still requires all three tasks; falsely marking them finished would permit a violating plant-enabled out_1 .

More generally, every run from all 126 entries traverses the same thirteen-start chain. At each start the source invariant holds; the remaining starts preserve it until the verified endpoint takes over. This establishes all thirteen new ACT obligations for this saved policy (Proposition L.1, Technical Appendix L.1). No ordinary event occurs on that suffix. This covers its reached tuples, not all initializer tuples or all 138 occurrences. The displayed path separately supplies one nonempty activation-safe history; GSM demonstrates active ordinary behavior (Section 5.1).

All-entry history boundary. Only 39/126 entries admit a safe full prefix; 87 do not (21 before entry, 66 during update). All entries admit an earlier violating cycle restoring zero observers. These limit A/H, not ACT. Replay covers 348 old/522 policy edges, not the compiler or safe-history nonemptiness for 38,288 initializer tuples. Runtime remains unchecked (Technical Appendix L).

PC2-Rolling: fixed-merge obstruction. The case in Section 6.2.1 yields fine-grained WIN. All checked old entries admit both raw transfers. Before their mandatory joint transfer, old plants lack calibration actions; ordinary moves and boundaries preserve readiness even after releasing old control. Every joint outcome calibrates both arms and clears both interval readiness bits, reaching absorbing error. This source argument proves impossibility independently of solver completion. The merged solver and fine Direct-Full nevertheless time out at 7200 s/200 GiB; these are not losing decisions.

Inherited inputs and additional witnesses. Merging transfers, requirement boundaries or both changes 0/54 Base/R1 decisions on the nine inherited inputs. Their total raw transfers omit the explicit startup/readiness phase creating the Rolling obstruction; totality alone does not guarantee safe joint transfer. States decrease in 44 comparisons and stay equal in 10; R2 names individual events and is excluded.

Additional restricted-transfer comparisons include both-LOSS and incomplete cases. Technical Appendix J gives aggregates and finite witnesses, with complete records in S1–S4; these populations are not pooled above.

6.3 RQ2: Do Policies Satisfy the Contract?

Winning and losing applications. Railcab retains both transfer outcomes, memories and targets. Its Base/R1 exports pass certificate and Link checks (branches in S4); runtime remains unvalidated.

Industry/R1 forbids validation between old-stop and new-start and loses at 42/131 entries under independent exhaustive checking. At its trap, GF1/TOR validation violates new requirements; DSD1 permits an infinite rejection cycle or deadlock if disabled. The full losing closure excludes all orders. Native DUCS and its aligned fork instead win on the source model with a subsequent-control objective [25, Defs. 4.1–4.3]; the adapted problems differ. Exempting GF1 yields all 131 WIN through earlier paths, not trap escape. Technical Appendix J retains details; neither check supplies timing.

Answer. Lazy and Direct-Full match all expected results from the independent raw labeled transition systems (LTSS).

Independent reconstruction and certificates. Python independently constructs graphs, ranks and losing closure for 46 fixtures: $46 \times 2 = 92$ jobs yield 36 WIN, 48 LOSS and 8 invalid (missing events, deadlocks or suppressed UC). All 84 eligible jobs pass certificate/exhaustive checks; all WIN jobs pass Link. Explicit successors are separate; frontend and endpoint/goal code are shared. These jobs are not independent applications.

Additional target, quiescence and initializer-lookup checks match independent reconstruction (Technical Appendix G); they are excluded from performance statistics.

Checking initializer interpretation. Table 3 distinguishes the 138 NEW and 135 UPD checks from the direct ACT arguments. Empty scopes pass vacuously; inclusion does not establish intent. Technical Appendix G gives the formula, label, reset and automaton checks and their complete counts.

The 25 returned WIN policies contain 8–1,071 states and maximum ranks 4–34, consistent across all five repeats. Ranks bound observed events, not downtime or minimum update length (Technical Appendix J.8).

6.4 RQ3: Does Lazy Reduce Exploration?

Answer. Lazy avoids states and queries in all 14 completed Direct-Full pairs. Within the fixed budget it decides 26 contracts, Eager 17 and Direct-Full 14; all paired decisions agree (Table 5). Update-first is faster on 8/24 paired solver medians but adds two timeouts. Lazy returns LOSS for Industry/R1 and times out on PC2/R2 at 1,200 s/64 GiB (fixed), 3,600 s/64 GiB (sensitivity) and 7,200 s/200 GiB (artifact appendix, earlier series); the limiting resource is unknown.

The separate Legacy reference yields 9 WIN, 13 OOM and 5 unmeasured snapshot-cap cases. Its subsequent-control objective differs from fixed-endpoint synthesis, so these are neither same-game performance ratios nor impossibility results (Technical Appendix J).

Table 5. All 27 contracts per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Pairs	Faster than Lazy	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—	—
Eager	17/0/10	17	0	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	24	8	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	14	0	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains its interruption and later same-setting timeout (S4).

Comparator/Lazy ratios are median [min–max] across completed pairs of five-run medians. Faster counts lower comparator solver time. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs have no ratio.

Figure 5 plots all 14 completed pairs’ absolute state/solver costs; Appendix D lists queries. Median Direct-Full/Lazy first-trial state/query ratios are 5.00/18.73. Base has the largest state savings; different completed populations do not isolate interval effects (Technical Appendix J).

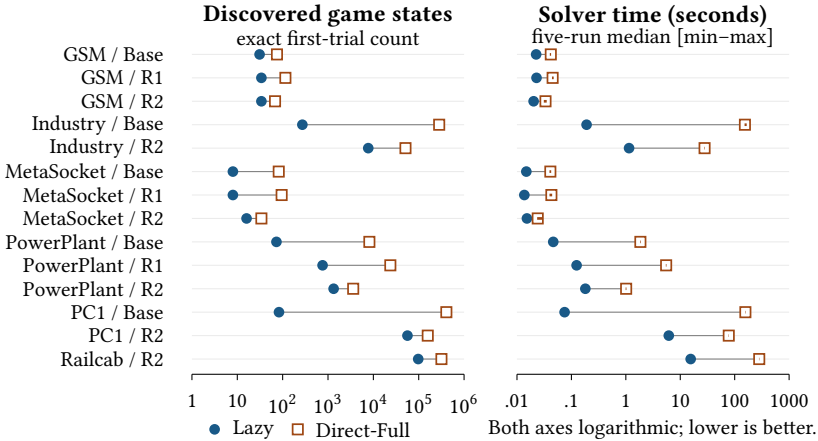


Fig. 5. Absolute costs for all 14 completed Lazy/Direct-Full pairs. Each row is WIN/WIN; Lazy explores fewer states and uses less solver time, with varying gaps. Solver time includes checking but excludes frontend, endpoint preparation and JVM startup. The other 13 Direct-Full contracts time out (Figure A2); they have no point here. This compares complete same-game procedures, not one isolated search factor.

Capped Direct-Full attempts give total-JVM lower bounds, not solver ratios or timing medians (Technical Appendix J.9).

Separating construction choices. Full+UC cannot isolate pruning time because builds, hosts and repetitions differ; no unpruned OTF is implemented (Technical Appendix J.9).

Scaling limits. All methods finish the independent family for $n = 2, \dots, 20$. At $n = 20$, Lazy and Update-first discover 21 states each, Eager 211, and Direct-Full 1,048,576 (S4).

Travel reaches timeouts and out-of-memory failures under all methods, including expanded-budget timeouts. These separate trials never enter medians; Technical Appendix J.9 retains all populations and adverse outcomes.

7 Related Work

DUCS already preserves legality and deadlock freedom and completes updates from every old-controller state [25, Defs. 4.1–4.3, Sec. 4.2], synthesizing update and subsequent control together. FG reuses the supplied new controller and preserves its continuation traces at the selected state (Proposition 3.2); Section 3.4 compares the interfaces.

PROTEOS filters states/interfaces before atomic replacement [12, Secs. 4.3–4.6]; workflow updating blocks tasks for safe updates [28, Secs. 4.3, 5.2]; Kitsune migrates whole C programs [13, Sec. 2]. FG-DUCS synthesizes ordinary steps, local transfers and requirement boundaries together; rank bounds events, not real time.

O-DUCS prioritizes updates and separates update/post-update exploration [15]. GR(1) updating preserves old then new safety and new justice under new-environment assumptions [2]; Live Synthesis retains unfinished obligations [11]. RS builds on requirement scopes [10] and residual rewriting [27].

State attraction bounds transitions to startup under weak invariance [26]. Dynamic-feature [30], compositional [22] and modular [33] control address specification structure. OTF-DCS requires marked extensions of controlled finite traces [8, Sec. 3]; FG requires finite quiescent handover on every maximal run without fairness. Performance comparison needs aligned objectives.

8 Threats to Validity

Construct and interpretation. Perpetual UC activity can prevent completion. Decoding, history completeness and A1–A4 remain unvalidated; resynthesis/repair are excluded. Witnesses assume finite startup, reporting, recovery and election; prevalence is unestablished. Only 39/126 PC2 entries have a safe full prefix for the selected new requirement.

Validation and measurement. Shared authorship, AI and code risk correlated faults; independent parsing covers restricted FSP. Direct-Full is in-house, Legacy’s objective differs, and pruning isolation is incomplete. External/compositional/symbolic solvers and randomized orders are untested. Ratios exclude unresolved costs; extended trials lack medians. Sampling may miss peaks; RSS establishes neither heap occupancy nor bottlenecks. S4 retains interruptions/follow-ups.

Generalization. Null/LOSS, faster Update-first, lower Eager/Update-first memory and Travel/PC limits bound gains; PC2’s merged timeout remains unresolved. Fixtures/repeats are not independent applications; timing uses one machine. Inherited inputs lack precedence; only Railcab transfers branch. Exponential separation assumes the default order.

9 Conclusion

FG-DUCS synthesizes safe, finite updates and preserves the selected new controller. Fine-grained choices yield 33/55 controlled separations; inherited inputs yield 0/54 differences. Lazy decides 26/27 contracts within budget and separates $n + 1$ from 2^n states under the default order. Deployment requires justified scopes and a conforming adapter.

Data Availability. The anonymous repository [4], snapshot fgducs-c55-20261002, provides the MTSA-derived source, all 27 application contracts, scripts, generated results and split raw-output archive. It includes this manuscript, its technical appendix and experimental supplement, complete inputs and source-history proofs, and saved PC2 activation analyses. The claim-to-evidence map gives file paths, saved-data checks and build commands, separating mathematical arguments from measured results and unvalidated assumptions. The package supports independent rebuilding of all three PDFs and checks of the saved evidence; these checks do not rerun performance experiments. Third-party dependencies are retrieved from official upstream sources.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Anonymous replication package. Manuscript and evidence snapshot, 2 October 2026; tag fgducs-c55-20261002. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c55-20261002>
- [5] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [6] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [7] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [8] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [9] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [10] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [11] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [12] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
- [13] Christopher M. Hayden, Karla Saur, Edward K. Smith, Michael Hicks, and Jeffrey S. Foster. 2014. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. *ACM Transactions on Programming Languages and Systems* 36, 4, Article 13 (Oct. 2014), 38 pages. doi:10.1145/2629460
- [14] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972

- [15] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. In Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/>
- [16] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [17] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [18] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [19] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [20] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
- [21] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [22] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
- [23] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [24] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS ’16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056
- [25] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [26] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [27] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [28] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE ’22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [29] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [30] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [31] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [32] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [33] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233